

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Зачётная работа №3
по дисциплине «Алгоритмы и структуры данных»
на тему «КОМБИНИРОВАННЫЕ СТРУКТУРЫ ДАННЫХ И
СТАНДАРТНАЯ БИБЛИОТЕКА ШАБЛОНОВ»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинко П.Г.

Санкт-Петербург
2025

Содержание

Цель работы	3
Задание	3
Описание контейнера и используемых функций STL.....	3
Результаты работы программы.....	4
Оценка временной сложности	7
Текст программы.....	8
Выводы	10
Список используемых источников.....	12

Цель работы

Реализовать пользовательскую структуру данных для хранения множеств с поддержкой операций над ними.

Задание

Реализовать структуру данных на базе дерева двоичного поиска с хранимой высотой (ДДПв) для хранения множеств и поддержки операций над последовательностями.

Необходимо выполнить цепочку операций над множествами по выражению: $A \cap B \cup C \cup (D \oplus E)$

А также реализовать следующие дополнительные операции над последовательностями:

MERGE — слияние с сохранением дубликатов,

EXCL — исключение подпоследовательности,

CHANGE — замена части последовательности с указанной позиции.

Результаты всех операций должны быть отображены на экране, а реализованные алгоритмы — устойчивы к исключениям.

Описание контейнера и используемых функций STL

Контейнер реализован в виде класса *Tree*, основанного на дереве двоичного поиска с хранимой высотой (ДДПв). Каждый узел дерева (*Node*) хранит:

- ключ (*key*),
- ссылки на левого и правого потомков ($L[2]$),
- высоту поддерева (*h*), которая используется для балансировки.

Для поддержки итерационного обхода реализован собственный итератор чтения *myiter*, а также итератор вставки *outiter*, что позволяет применять алгоритмы из *STL*.

В ходе выполнения цепочки операций над множествами использовались стандартные алгоритмы из заголовка *<algorithm>*:

- *set_intersection* — пересечение множеств ($A \cap B$);

- *set_union* — объединение множеств ($A \cup B$);
- *set_symmetric_difference* — исключающее или (симметрическая разность) ($A \oplus B$).

Каждый алгоритм использует итераторы контейнера *Tree* и записывает результат в новый экземпляр с помощью *outserter(...)*.

Также реализованы пользовательские шаблонные функции для выполнения операций над последовательностями, представленными векторами типа *std::vector<int>*. Эти функции позволяют продемонстрировать работу с произвольными последовательностями и использовать результаты для последующего преобразования в дерево.

- *MERGE(s1, s2)* — выполняет слияние двух отсортированных последовательностей с сохранением всех элементов, включая дубликаты. Результирующая последовательность содержит элементы обеих входных последовательностей в упорядоченном виде.
- *EXCL(mainSeq, subSeq)* — осуществляет удаление подпоследовательности *subSeq* из основной последовательности *mainSeq*, если такая подпоследовательность встречается как непрерывный фрагмент. Если совпадение найдено, соответствующий участок удаляется из основного вектора.
- *CHANGE(mainSeq, pos, subSeq)* — заменяет часть основной последовательности на другую, начиная с указанной позиции *pos*. При этом количество удаляемых элементов зависит от длины вставляемой подпоследовательности, и новая последовательность сохраняет корректный порядок элементов.

Результаты работы программы

Ниже приведены поэтапные результаты выполнения заданной цепочки операций над множествами и последовательностями. Каждая операция

сопровождается визуализацией дерева (BSTh) — отображением структуры полученного множества после преобразования.

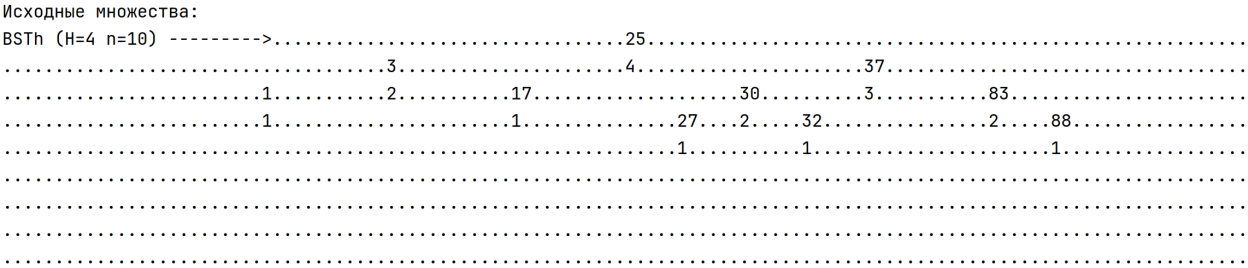


Рисунок 1 – Исходное множество *A*

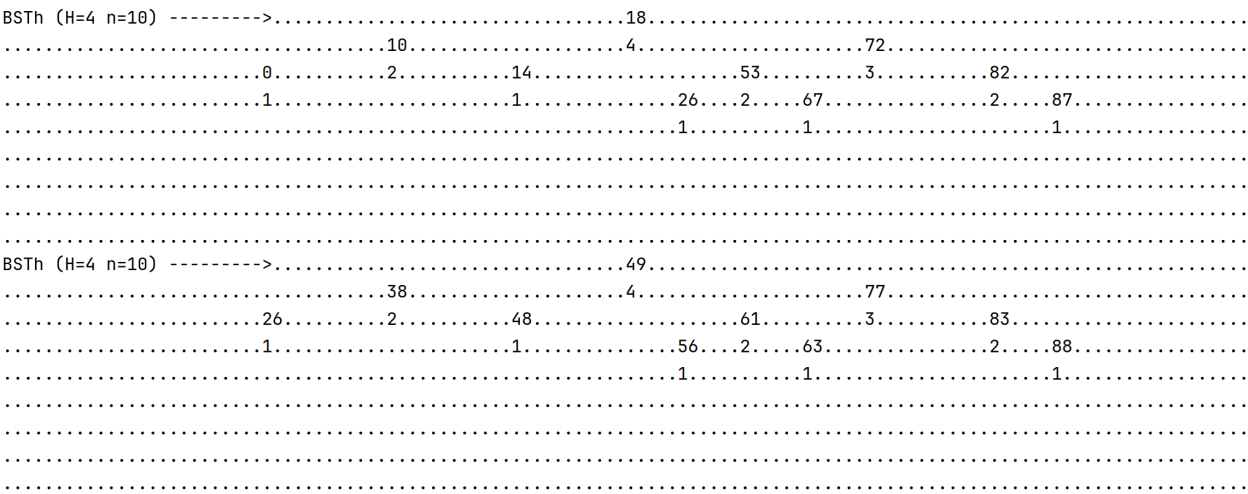


Рисунок 2 – Исходные множества *B* и *C*

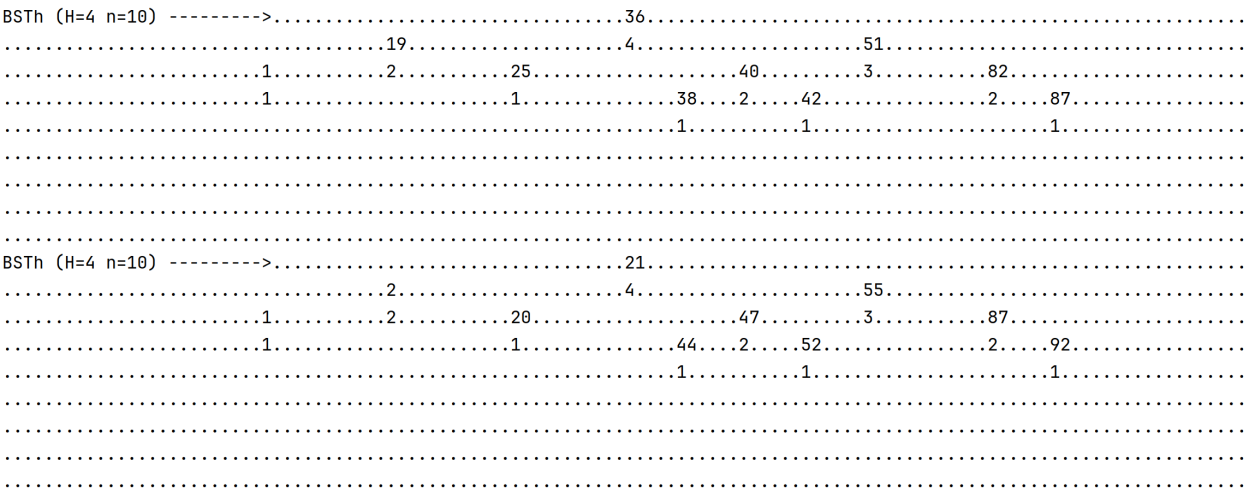
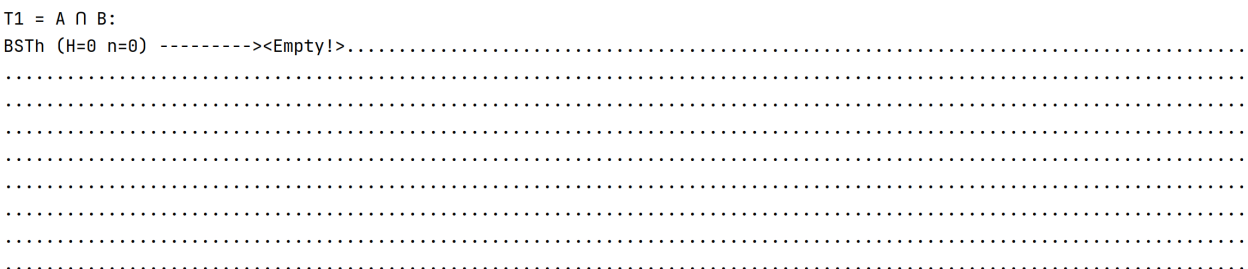


Рисунок 3 – Исходные множества *D* и *E*



```
T2 = T1 U C:
BSTh (H=4 n=10) ----->.....49.....
.....38.....4.....77.....
.....26.....2.....48.....61.....3.....83.....
.....1.....1.....56.....2.....63.....2.....88.....
.....1.....1.....1.....
.....
.....
.....
```

[illegible][illegible]

```
A: 1 3 17 25 27 30 32 37 83 88
B: 0 10 14 18 26 53 67 72 82 87
MERGE(A,B): 0 1 3 10 14 17 18 25 26 27 30 32 37 53 67 72 82 83 87 88
Дерево из MERGE:
BSTh (H=5 n=20) ----->.....25.....
.....10.....5.....72.....
.....1.....3.....17.....32.....4.....83.....
.....0...2...3.....14...2...18.....27...3...53.....82...3...87.....
.....1.....1.....1.....1.....26.2.30...37.2.67.....1.....2.88.....
.....1.....1.....1.....1.....1.....
.....
```

```

EXCL (удалили subseq [3 10 14 ]): 0 1 17 18 25 26 27 30 32 37 53 67 72 82 83 87 88
Дерево из EXCL:
BSTh (H=5 n=17) ----->. . . . .30. . . . .
. . . . .18. . . . .5. . . . .67. . . . .
. . . . .1. . . . .3. . . . .26. . . . .37. . . . .4. . . . .82. . . . .
. . . . .0. . . . .2. . . . .17. . . . .25. . . . .2. . . . .27. . . . .32. . . . .2. . . . .53. . . . .72. . . . .3. . . . .87. . . . .
. . . . .1. . . . .1. . . . .1. . . . .1. . . . .1. . . . .1. . . . .1. . . . .83. 2. 88. . . . .
. . . . . . . . . . .1. . . . .1. . . . .
. . . . .
. . . . .

```

Рисунок 9 – Удаление подпоследовательности {3,10,14}

CHANGE (с позиции 3 -> {100,200,300}): 0 1 3 100 200 300 18 25 26 27 30 32 37 53 67 72 82 83 87 88

Дерево из CHANGE:

```
BSTh (H=5 n=20) ----->.....26.....
.....18.....5.....67.....
.....1.....4.....25.....37.....5.....100.....
.....0.....2.....3.....1.....30.....4.....53.....82.....4.....200.....
.....1.....1.....27.....2.....32.....1.....72.....3.....87.....2.....300.....
.....1.....1.....1.....832.88.....1.....
.....1.....1.....
.....
```

Рисунок 10 – Замена с 3 позиции на {100,200,300}

Программа последовательно выполняет цепочку операций над множествами, включая пересечение, объединение и симметрическую разность, а также демонстрирует работу с последовательностями через пользовательские функции *MERGE*, *EXCL* и *CHANGE*. На всех этапах результаты визуализируются в виде сбалансированных деревьев, что позволяет наглядно убедиться в корректности работы алгоритмов.

Оценка временной сложности

Реализация операций над множествами основана на использовании стандартных алгоритмов из библиотеки *STL* (*set_union*, *set_intersection*, *set_symmetric_difference*). Благодаря использованию итераторов и упорядоченности элементов во внутреннем обходе дерева, данные алгоритмы работают со средней временной сложностью $O(n)$, где n — общее количество элементов во входных множествах.

Операции вставки и удаления элементов в дерево двоичного поиска с хранимой высотой имеют среднюю сложность $O(\log n)$. Благодаря балансировке по высоте, дерево остаётся сбалансированным, что предотвращает вырождение и сохраняет эффективность операций. В худшем случае (при нарушении баланса) сложность может достигать $O(n)$.

Дополнительные операции реализованы с использованием стандартного контейнера *std::vector*. Ниже приведены краткие характеристики каждой:

- *MERGE* — выполняется с помощью функции *std::merge*, объединяющей два отсортированных вектора в третий с сохранением

всех элементов, включая дубликаты. Сложность: $O(n + m)$, где n и m — размеры входных последовательностей.

- *EXCL* — реализована через *std::search* для поиска подпоследовательности в векторе, затем используется *erase* для её удаления. Поиск подпоследовательности: $O(n \cdot m)$ в худшем случае (наивный алгоритм), где n — длина основного вектора, m — длина удаляемой части. В среднем случае, при редких совпадениях, — ближе к $O(n)$.
- *CHANGE* — сочетание операций *erase* и *insert* на указанный диапазон. Удаление и вставка элементов с конца происходит за амортизированное $O(k)$, где k — количество заменяемых элементов. Общая сложность: $O(k + m)$, где k — удаляемое подмножество, m — вставляемое.

Текст программы

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <ctime>
#include "BSTh.h"

using namespace std;

// Генерация упорядоченного вектора из 10 уникальных случайных чисел <100
static vector<int> genSet() {
    vector<int> s;
    while (s.size() < 10) {
        int x = rand() % 100;
        if (find(s.begin(), s.end(), x) == s.end())
            s.push_back(x);
    }
    sort(s.begin(), s.end());
    return s;
}

// MERGE: слияние двух упорядоченных последовательностей с сохранением дубликатов
template<typename Seq>
Seq MERGE(const Seq& s1, const Seq& s2) {
    Seq res;
    merge(s1.begin(), s1.end(), s2.begin(), s2.end(), back_inserter(res));
    return res;
}

// EXCL: удаление подпоследовательности из основной
template<typename Seq>
Seq EXCL(const Seq& mainSeq, const Seq& subSeq) {
```

```

    Seq res(mainSeq);
    auto it = search(res.begin(), res.end(), subSeq.begin(), subSeq.end());
    if (it != res.end())
        res.erase(it, next(it, subSeq.size()));
    return res;
}

// CHANGE: замена части последовательности на другую с заданной позиции
template<typename Seq>
Seq CHANGE(const Seq& mainSeq, size_t pos, const Seq& subSeq) {
    Seq res(mainSeq);
    if (pos <= res.size()) {
        auto it = next(res.begin(), pos);
        res.erase(it, next(it, min(subSeq.size(), res.size() - pos)));
        res.insert(it, subSeq.begin(), subSeq.end());
    }
    return res;
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(static_cast<unsigned>(time(nullptr)));

    // 1. Генерация пяти множеств A, B, C, D, E
    auto A = genSet();
    auto B = genSet();
    auto C = genSet();
    auto D = genSet();
    auto E = genSet();

    Tree tA(A.begin(), A.end()),
        tB(B.begin(), B.end()),
        tC(C.begin(), C.end()),
        tD(D.begin(), D.end()),
        tE(E.begin(), E.end());

    cout << "Исходные множества:\n";
    tA.Display(); tB.Display(); tC.Display(); tD.Display(); tE.Display();

    // 2. T1 = A ∩ B
    Tree T1;
    set_intersection(tA.begin(), tA.end(), tB.begin(), tB.end(),
                    outinserter(T1, myiter(nullptr)));
    cout << "\nT1 = A ∩ B:\n"; T1.Display();

    // 3. T2 = T1 ∪ C
    Tree T2;
    set_union(T1.begin(), T1.end(), tC.begin(), tC.end(),
              outinserter(T2, myiter(nullptr)));
    cout << "\nT2 = T1 ∪ C:\n"; T2.Display();

    // 4. T3 = D ⊕ E
    Tree T3;
    set_symmetric_difference(tD.begin(), tD.end(),
                             tE.begin(), tE.end(),
                             outinserter(T3, myiter(nullptr)));
    cout << "\nT3 = D ⊕ E:\n"; T3.Display();

    // 5. Итог = T2 ∪ T3
    Tree T;
    set_union(T2.begin(), T2.end(), T3.begin(), T3.end(),

```

```

        outinserter(T, myiter(nullptr));
cout << "\nИТОГ = T2 U T3:\n"; T.Display();

// --- Доп. операции над последовательностями ---
cout << "\nA: "; for (int x : A) cout << x << " "; cout << "\n";
cout << "B: "; for (int x : B) cout << x << " "; cout << "\n";

auto merged = MERGE(A, B);
cout << "MERGE(A,B): ";
for (int x : merged) cout << x << " "; cout << "\n";
Tree tM(merged.begin(), merged.end());
cout << "Дерево из MERGE:\n"; tM.Display();

vector<int> subExcl(merged.begin() + 2, merged.begin() + 5);
auto excl = EXCL(merged, subExcl);
cout << "EXCL (удалили subseq [";
for (int x : subExcl) cout << x << " ";
cout << "]): ";
for (int x : excl) cout << x << " "; cout << "\n";
Tree tE2(excl.begin(), excl.end());
cout << "Дерево из EXCL:\n"; tE2.Display();

vector<int> subChange{100,200,300};
auto changed = CHANGE(merged, 3, subChange);
cout << "CHANGE (с позиции 3 -> {100,200,300}): ";
for (int x : changed) cout << x << " "; cout << "\n";
Tree tC2(changed.begin(), changed.end());
cout << "Дерево из CHANGE:\n"; tC2.Display();

return 0;
}

```

Выводы

В результате выполнения работы была реализована структура данных — дерево двоичного поиска с хранимой высотой (ДДПв), предназначенная для хранения и обработки множеств. Структура обеспечивает эффективную вставку, удаление и поиск элементов за логарифмическое время в среднем случае благодаря поддержке сбалансированного состояния дерева.

Была выполнена цепочка операций над множествами согласно индивидуальному заданию: пересечение множеств A и B , объединение результата с множеством C , вычисление симметрической разности (XOR) D и E , и объединение промежуточных результатов. Для реализации этих операций использовались стандартные алгоритмы библиотеки *STL* (*set_intersection*, *set_union*, *set_symmetric_difference*) с применением пользовательских итераторов для совместимости с алгоритмами. Это позволило организовать

обработку множеств кратко, эффективно и с использованием стандартных подходов.

Дополнительно были реализованы и протестированы операции над последовательностями:

- *MERGE* — слияние двух отсортированных последовательностей с сохранением дубликатов;
- *EXCL* — удаление подпоследовательности из основной, если она встречается как непрерывный фрагмент;
- *CHANGE* — замена части последовательности другой с указанной позиции.

Эти операции выполнялись над контейнерами типа *std::vector*, что позволило гибко управлять упорядоченными наборами данных. После каждой операции над последовательностями результат отображался как в виде вектора, так и в виде дерева, созданного на его основе, что позволило проанализировать влияние последовательных изменений на структуру данных.

Проведённый анализ показал, что:

- операции над множествами выполняются в среднем за $O(n)$ благодаря работе с упорядоченными структурами и применению эффективных *STL*-алгоритмов;
- операции вставки и удаления в дерево в среднем имеют сложность $O(\log n)$;
- операции над последовательностями работают за линейное время в большинстве случаев, исключая редкие ситуации с полным перебором (например, поиск подпоследовательности в *EXCL*).

Программа успешно обрабатывает случайные данные заданной мощности, корректно выполняет все требуемые операции и отображает промежуточные результаты.

Список используемых источников

1. Колинъко П. Г. Пользовательские контейнеры: учебно-метод. пособие. СПб.: СПбГЭТУ «ЛЭТИ», 2025. 64 с. (вып.2502)